Introduction

CASE tools known as Computer-aided software engineering tools is a kind of component-based development which allows its users to rapidly develop information systems. The main goal of case technology is the automation of the entire information systems development life cycle process using a set of integrated software tools, such as modelling, methodology and automatic code generation. Component based manufacturing has several advantages over custom development. The main advantages are the availability of high quality, defect free products at low cost and at a faster time. The prefabricated components are customized as per the requirements of the customers. The components used are pre-built, ready-tested and add value and differentiation by rapid customization to the targeted customers. However the products we get from case tools are only a skeleton of the final product required and a lot of programming must be done by hand to get a fully finished, good product. Characteristics of CASE:

Some of the characteristics of case tools that make it better than customized development are;

It is a graphic oriented tool.

It supports decomposition of process. Some typical CASE tools are:
Unified Modeling Language

Data modeling tools, and

Source code generation tools

UNIFIED MODELING LANGUAGE

Introduction

The unified modeling language (UML) is a standard language for writing software blue prints of the system.

Definition:

The UML is a language for

- Visualizing
- Specifying
- Constructing
- Documenting

the artifacts of a software system.

- UML is a language that provides vocabulary and the rules for combing words in that vocabulary for the purpose of communication.
- Vocabulary and rules of a language tell us how to create and real well formed models, but they don‟t tell you what model you should create and when should create them.

Building Blocks of the UML:

The vocabulary of the UML encompasses three kinds of building blocks:

1. Things
2. Relationships

3. Diagrams

Things are abstractions that are first-class citizens in a model; Relationships tie these things together;
Diagrams group interesting collections of things.

Things

Things are the most important building blocks of UML. There are four kinds of things in the UML.

1. Structural things
2. Behavioral things
3. Grouping things
4. Annotational things

1) STRUCTURAL THINGS:

Structural things are the nouns of the UML models.

These are static parts of the model, representing elements that are either conceptual or physical.

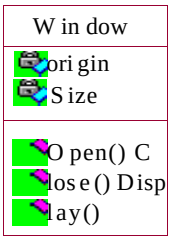
There are seven kinds of Structural things.

1. Class
2. Interface
3. Collaboration
4. Use case
5. Active class
6. Component
7. Node

Class:

A class is a description of a set of objects that shares the common attributes, operations, relationships, and semantics. A class implements one or more interfaces.

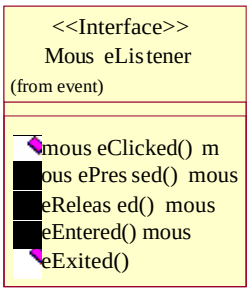
Graphically, a class is represented as a rectangle, usually including its name, attributes and operations, as shown below.



Interface:

An interface is a collection of operations that specify a service of a class or component. An interface describes the externally visible behavior of that element.

Graphically the interface is rendered as a circle together with its name.

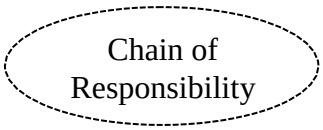


I Window

Collaboration:

Collaboration defines an interaction and is a society of roles and other elements that work together to provide some cooperative behavior that"s bigger than the sum of all the elements.

Graphically, collaboration is rendered as an ellipse with dashed lines, usually including only its name as shown below.



UseCase:

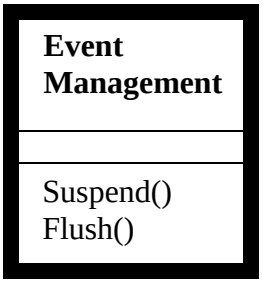
Use case is a description of a set of sequence of actions performed by a system for a specific goal for the system.

Graphically, Use Case is rendered as an ellipse with dashed lines, usually including only its name as shown below.

ActiveClass:

An active class is a class whose objects own one or more processes or threads and therefore can initiate control activity.

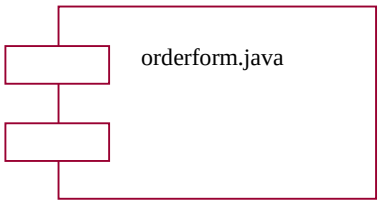
Graphically, an active class is rendered just like a class, but with heavy lines usually including its name, attributes and operations as shown below.



Component:

Component is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces.

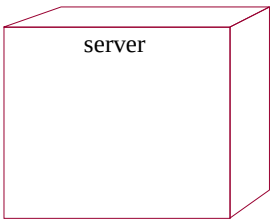
Graphically, a component is rendered as a rectangle with tabs, usually including only its name, as shown below.



Node:

A Node is a physical element that exists at run time and represents a computational resource, generally having at least some memory and often, processing capability.

Graphically, a node is rendered as a cube, usually including only its name, as shown below.



2) BEHAVIORAL THINGS:

Behavioral things are the dynamic parts of UML models.

These are the verbs of a model, representing behavior over time and space.

1) ***Interaction:***

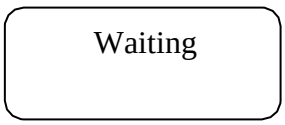
An interaction is a behavior that consists of a set of messages exchanged among a set of objects(elements) within a particular context to accomplish a specific task.

Graphically, a message is rendered as a direct line, almost always including the name if its operation, as shown below.

2) ***State Machine:***

A state machine is a behavior that specifies the sequence of states of an object in its life cycle. It defines the sequence of states an object goes through in response to events.

Graphically, a state is rendered as a rounded rectangle usually including its name and its sub-states, if any, as shown below.



3) **GROUPING THINGS:**

Grouping things are the organizational parts of the UML models. These are the boxes into which a model can be decomposed.

There is one primary kind of grouping thing with “package”.

Package:

A package is a general-purpose mechanism for organizing elements into groups.

Package is the only one grouping thing available for gathering structural and behavioral things.

Package

4) **ANNOTATIONAL THINGS:**

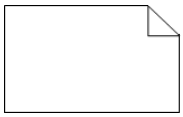
Annotational things are the explanatory parts of the UML models.

Annotational things can be defined as a mechanism to capture remarks, descriptions, and comments of UML model elements.

Note:

A note is simply a symbol for rendering constraints and comments attached to an element or a collection of elements.

Graphically a note is represented as a rectangle with dog-eared corner together, with a textual or graphical comment, as shown below.



Note

RELATIONSHIPSINTHEUML:

Relationship is another most important building block of UML. It shows how elements are associated with each other and this association describes the functionality of an application. There are four kinds of relationships in the UML:

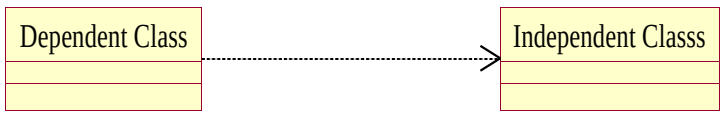
1. Dependency
2. Association
3. Generalization
4. Realization

Dependency

Dependency is a relationship between two things in which change in one element also affects the other one.

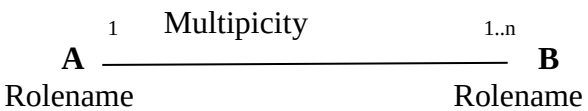
Dependency

Ex:



Association:

Association is basically a set of links that connects elements of an UML model. It also describes how many objects are taking part in that relationship.



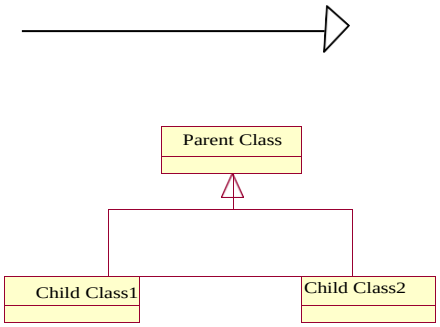
Association

Ex:

Generalization:

Generalization can be defined as a relationship which connects a specialized element with a generalized element. It basically describes inheritance relationship in the world of objects.

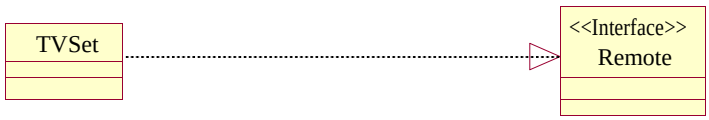
Ex:



Realization:

Realization can be defined as a relationship in which two elements are connected. One element describes some responsibility which is not implemented and the other one implements them. This relationship exists in case of interfaces.

Ex:



DIAGRAMSINUML:

All the elements, relationships are used to make a complete UML diagram and the diagram represents a system.

The visual effect of the UML diagram is the most important part of the entire process.

Each UML diagram is designed to let developers and customers view a software system from a different perspective and in varying degrees of abstraction.

UML diagrams are the ultimate output of the entire system.

A diagram is the graphical presentation of a set of elements ,most often rendered as a connected graph of vertices(things) arcs (relationships).

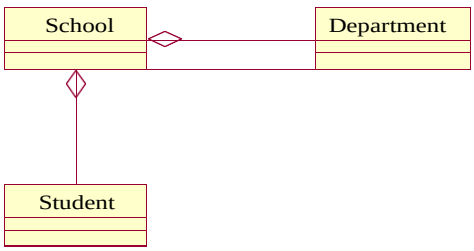
UML includes the following nine diagrams:

1) Class diagram
2) Object diagram
3) Use case diagram
4) Sequence diagram
5) Collaboration diagram
6) Activity diagram
7) State chart diagram
8) Deployment diagram
9) Component diagram

1.ClassDiagram

Class diagram is a diagram that shows a set of classes, interfaces, and collaborations and their relationships. Class diagrams address the static design view or the static process view of the system.

Graphically it is represented as follows:-



2. ObjectDiagram

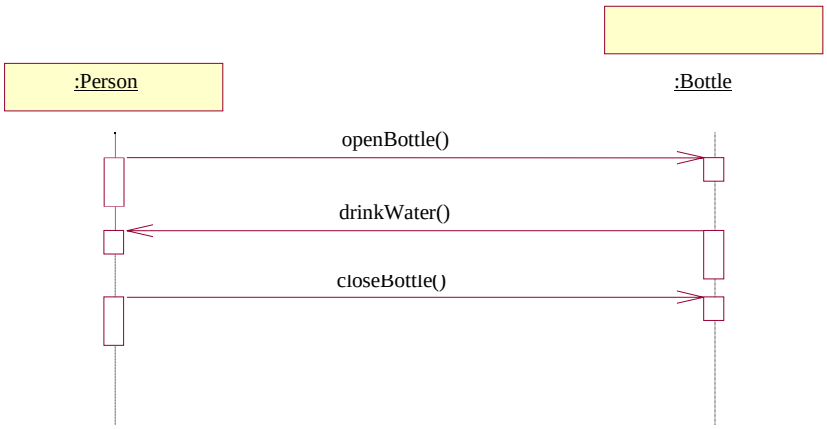
Object diagram shows a set of objects and their relationships. These diagram the static design view or static process view of a system.

3. UsecaseDiagram

Use Case diagram shows a set of use cases and actors (a special kind of class) and their relationships. These diagrams address the static use case view of a system. Graphically it is represented as follows:-

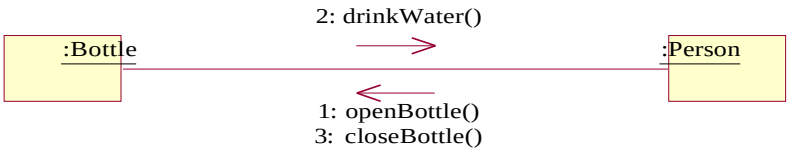
4.SequenceDiagram

Sequence diagram are interaction diagrams. This diagram emphasizes the time-ordering of messages. These diagrams address the dynamic view of a system. Sequence Diagram displays the time sequence of the objects participating in the interaction. This consists of the vertical dimension (time) and horizontal dimension (different objects).Graphically it is represented as
follows:-



5 CollaborationDiagram

Collaboration diagram are also interaction diagrams. These diagrams emphasizes the structural organization of the objects that send and receive messages. These diagrams address the dynamic view of a system. Collaboration Diagram displays an interaction organized around the objects and their links to one another. Numbers are used to show the sequence of messages.Graphically it is
represented as follows:-



6. StatechartDiagram

State chart diagram shows a state machine, consisting of states, transitions, events and activities. These diagrams address the dynamic view of the system. State Chart diagram displays the sequences of states that an object of an interaction goes through during its life in response to received stimuli, together with its responses and actions.

7 .ActivityDiagram

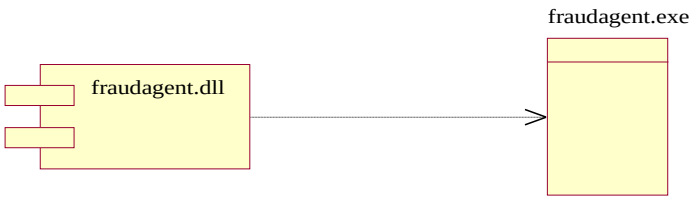
Activity diagram is a special kind of a state chart diagram that shows the flow from activity to activity within a system. These diagrams address dynamic view of a system. Activity Diagram displays a special

state diagram where most of the states are action states and most of the transitions are triggered by completion of the actions in the source states. Graphically it is represented as follows:-

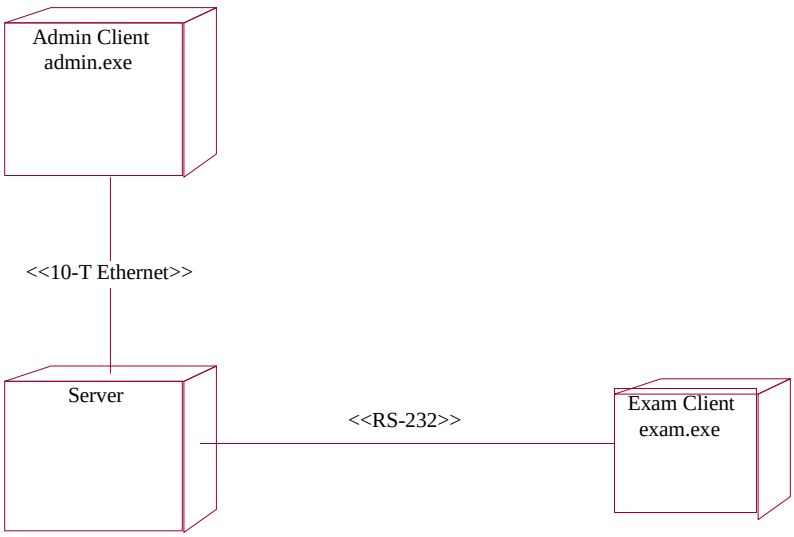
8. <u>ComponentDiagram</u>

Component diagram shows the organizations and dependencies among a set of components. These diagrams address the static implementation of view of a system. Component Diagram displays the high level packaged structure of the code itself. Dependencies among components are shown, including source code components, binary code components, and executable components. Some components exist at compile time, at link time, at run times well as at more

than one time.Graphically it is represented as follows:-



9. <u>DeploymentDiagram</u>

Deployment diagram shows the configuration of run-time processing nodes and the components that live on them. These diagrams address the static deployment view of architecture. Deployment Diagram displays the configuration of run-time processing elements and the software components, processes, and objects that live on them. Software component instances represent run-time manifestations of code.

Graphically it is represented as follows:-